To add the CORS services to your application, call `AddCors()` on your `WebApplicationBuilder` instance in Program.cs:

```
builder.Services.AddCors();
```

The bulk of your effort in configuring CORS will go into policy configuration. A CORS policy controls how your application responds to cross-origin requests. It defines which origins are allowed, which headers to return, which HTTP methods to allow, and so on. You normally define your policies inline when you add the CORS services to your application.

Consider the previous e-commerce site example. You want your API that is hosted at http://api.shopping.com to be available from the main app via client-side JavaScript, hosted at http://shopping.com. You therefore need to configure the *API* to allow cross-origin requests.

NOTE Remember, it's the app that will get errors when attempting to make cross-origin requests, but it's the API you're accessing that you need to add CORS to, not the app making the requests.

The following listing shows how to configure a policy called `"AllowShoppingApp"` to enable cross-origin requests from http://shopping.com to the API. Additionally, we explicitly allow any HTTP verb type; without this call, only simple methods (`GET`, `HEAD`, and `POST`) are allowed. The policies are built up using the familiar fluent builder style you've seen throughout this book.

Listing 29.2 Configuring a CORS policy to allow requests from a specific origin

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddCors(options => {                          ❶
```